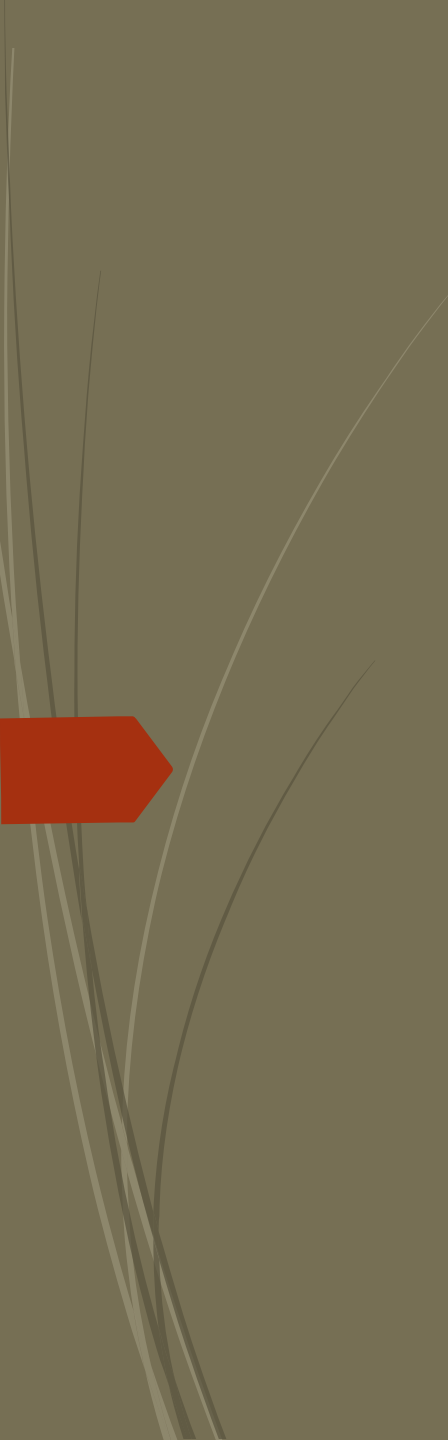




CS3006 – Parallel and Distributed Computing

□ Lecture # 01

□ Spring 2023

□ FAST – NUCES, Faisalabad Campus

Agenda

2

- **Introduction to Parallel and Distributed Computing**
- **Motivating Parallelism**
- **Computing vs Systems**
- **Parallel vs Distributed Computing**
- **Practical Applications of P&D Computing**
- **Organization and Contents of the Course**

Computing

3

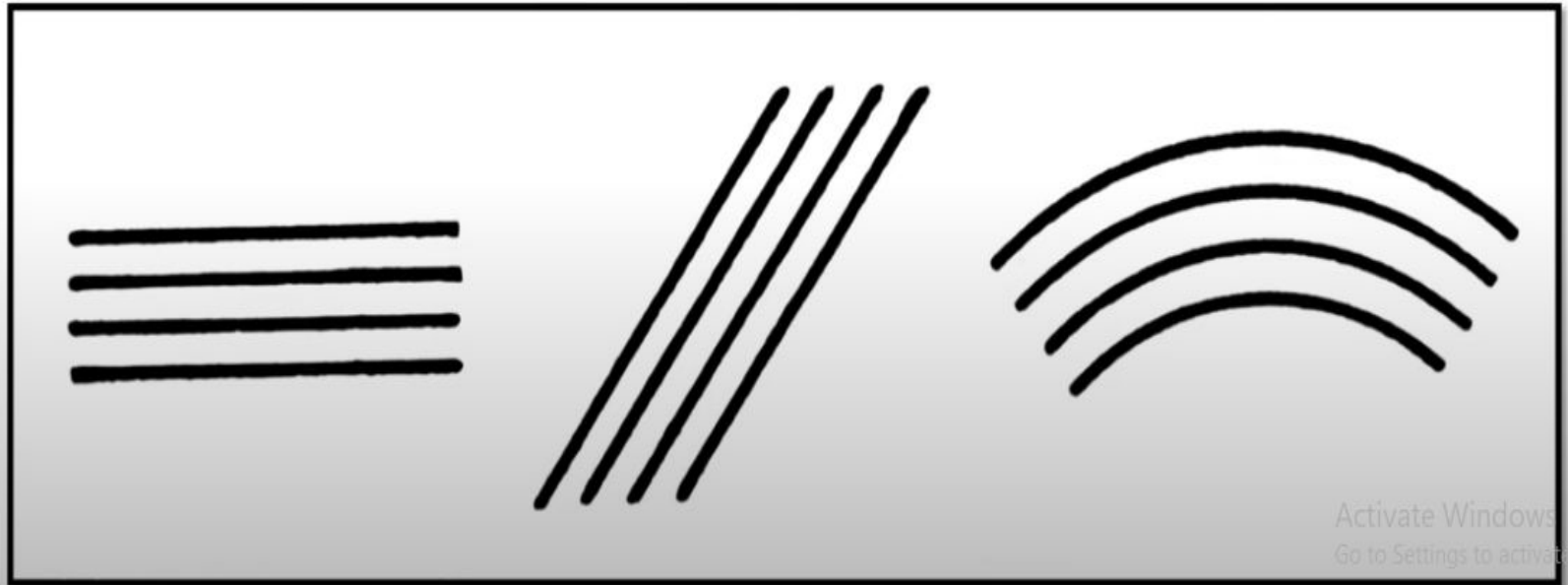
Computing

- Computing is the **process to complete a given goal-oriented task** by **using computer technology**.
- Computing may include the **design and development** of **software and hardware systems** for a **broad range of purposes**, often consist of ***structuring, processing*** and ***managing*** any kind of **information**.

Parallel

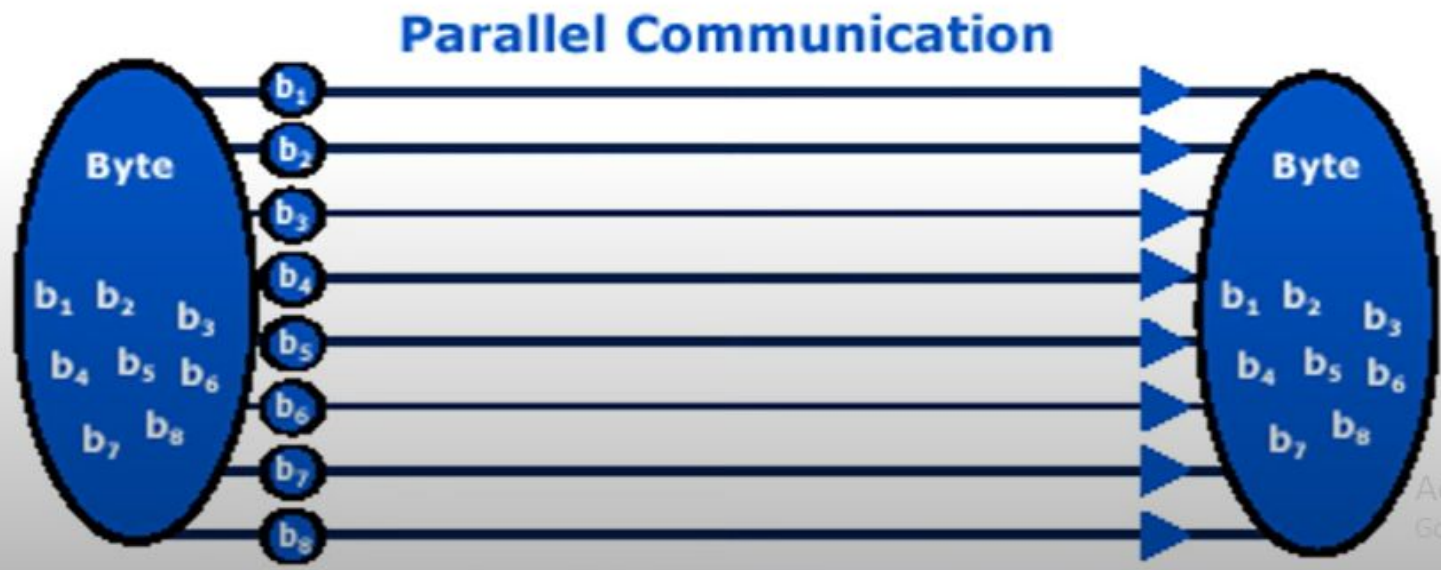
4

- Parallel (in mathematics) means two lines that never intersect — think of an equal sign.



Serial vs Parallel

5



Parallel Computing System

6

- **Parallel computing systems** are the **simultaneous execution** of the single task (split up and adapted) on multiple processors in order to obtain results faster.
- The idea is based on the fact that the process of solving a problem usually can be divided into **smaller tasks (divide and conquer)**, which may be carried out simultaneously with some coordination.

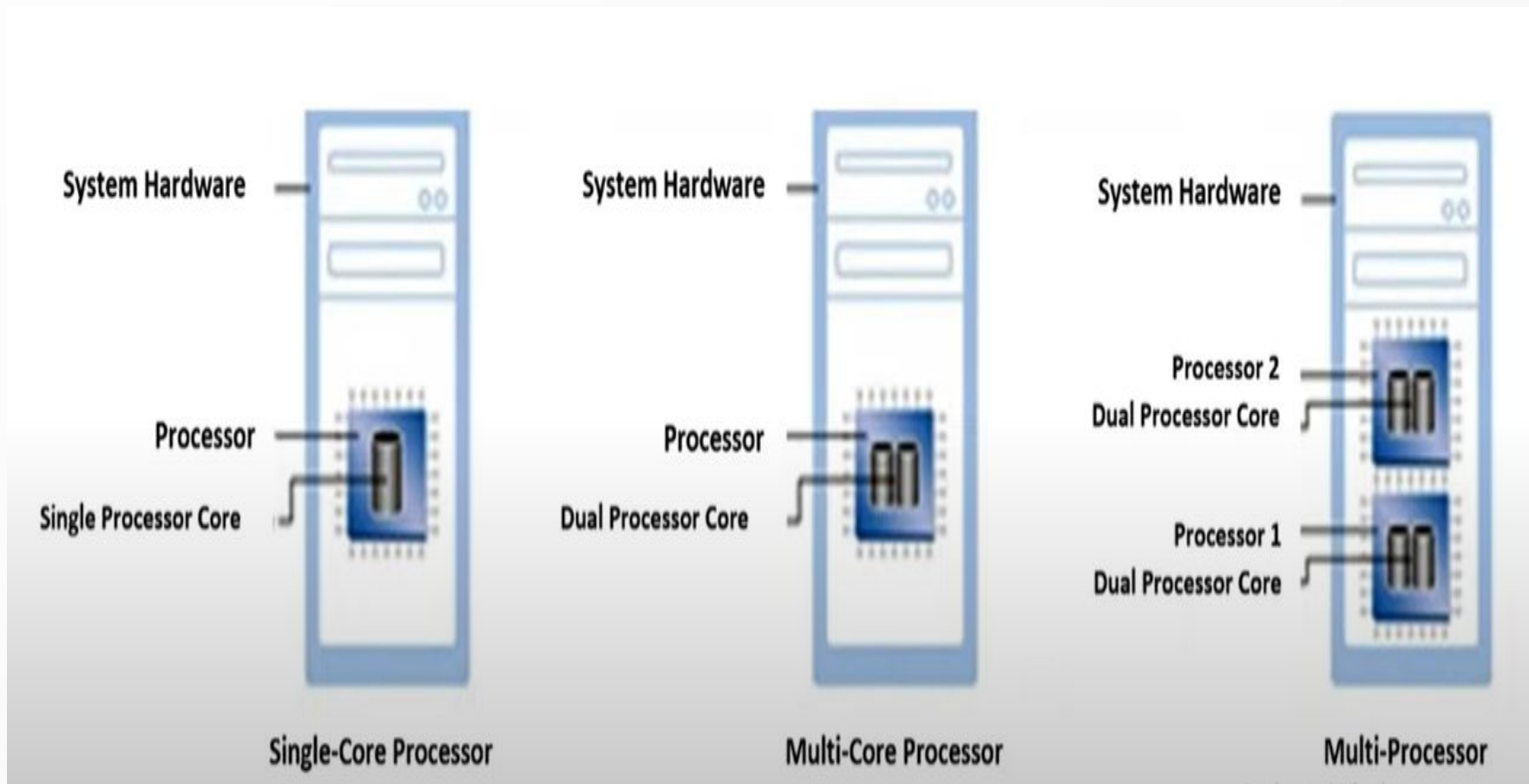
Parallel Computing System

7

- The terms **parallel computing architecture** sometimes used for a computer with **more than one processor** (few to thousands), available for processing.
- The recent **multicore processors** (chips with more than one processor core) are some commercial examples which **bring parallel computing to the desktop**.

Parallel Computing System

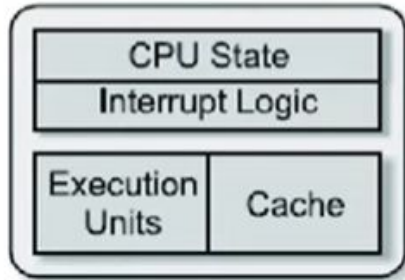
Multi-core and Multi-processor



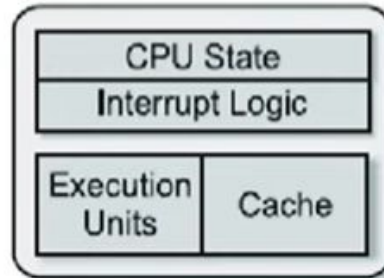
Parallel Computing System

9

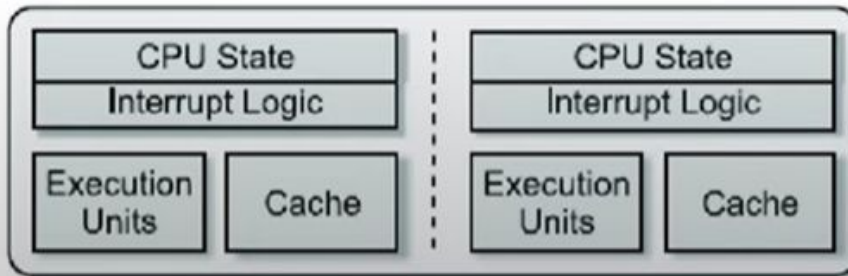
Multi-core and Multi-processor



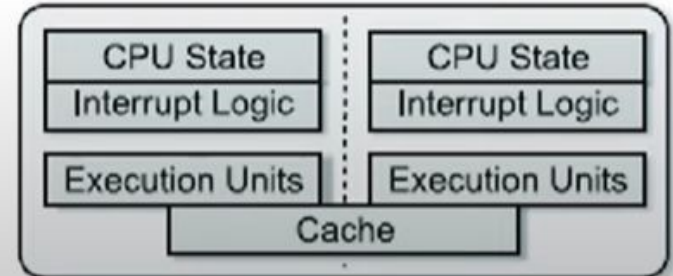
(a) Single core



(b) Multiprocessor



(c) Multi-core



(d) Multi-core with shared cache

Activate Wi
Core Settings

Distributed System

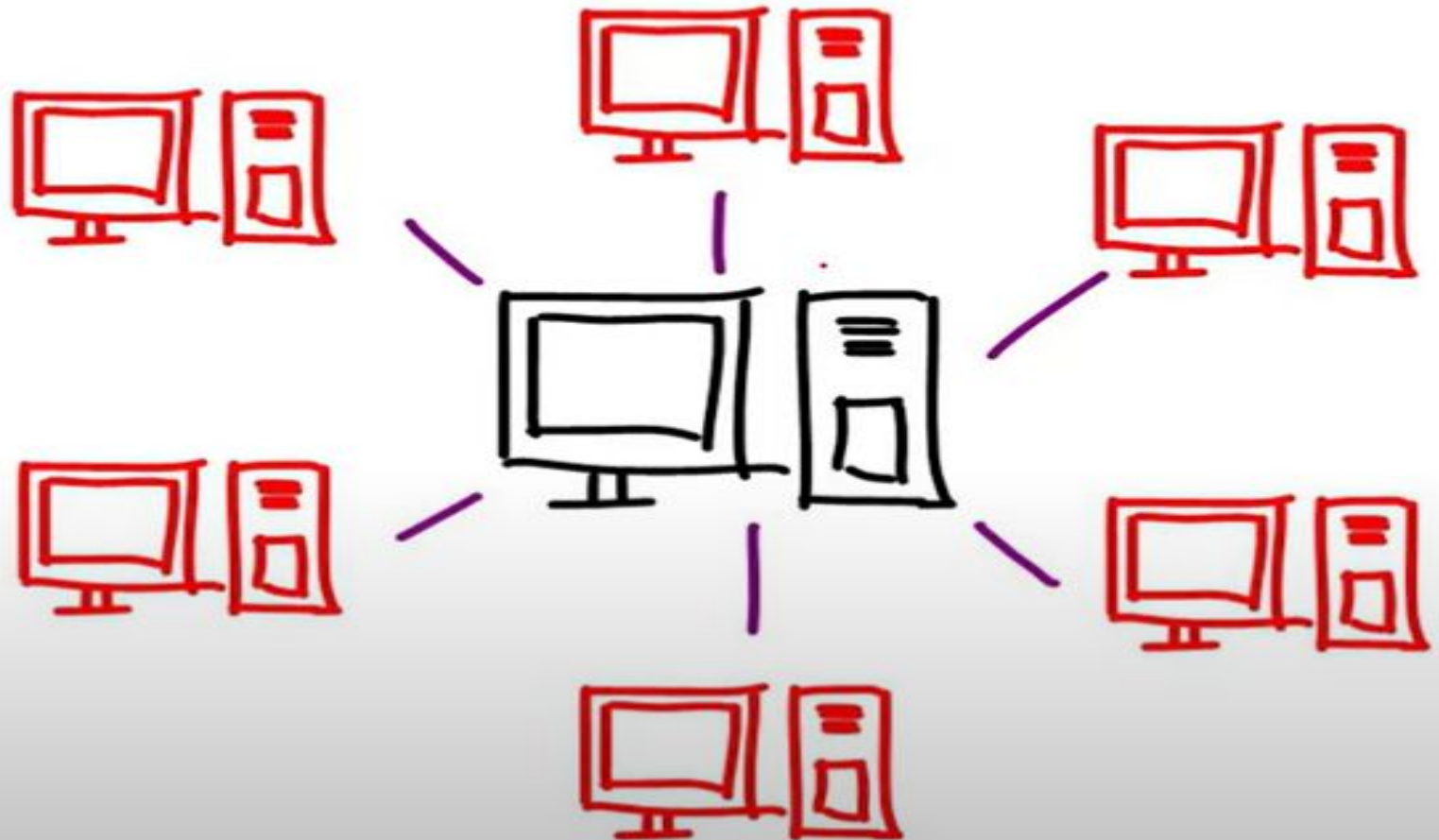
10

- We define a **distributed system** as one in which *hardware or software components* located at **networked computers** **communicate and coordinate their actions only by passing messages**.
- This simple definition covers the entire range of systems in which networked computers can usefully be deployed.
- Education System in Current scenario (online class).

Activate Windows
Go to Settings to activate Windows

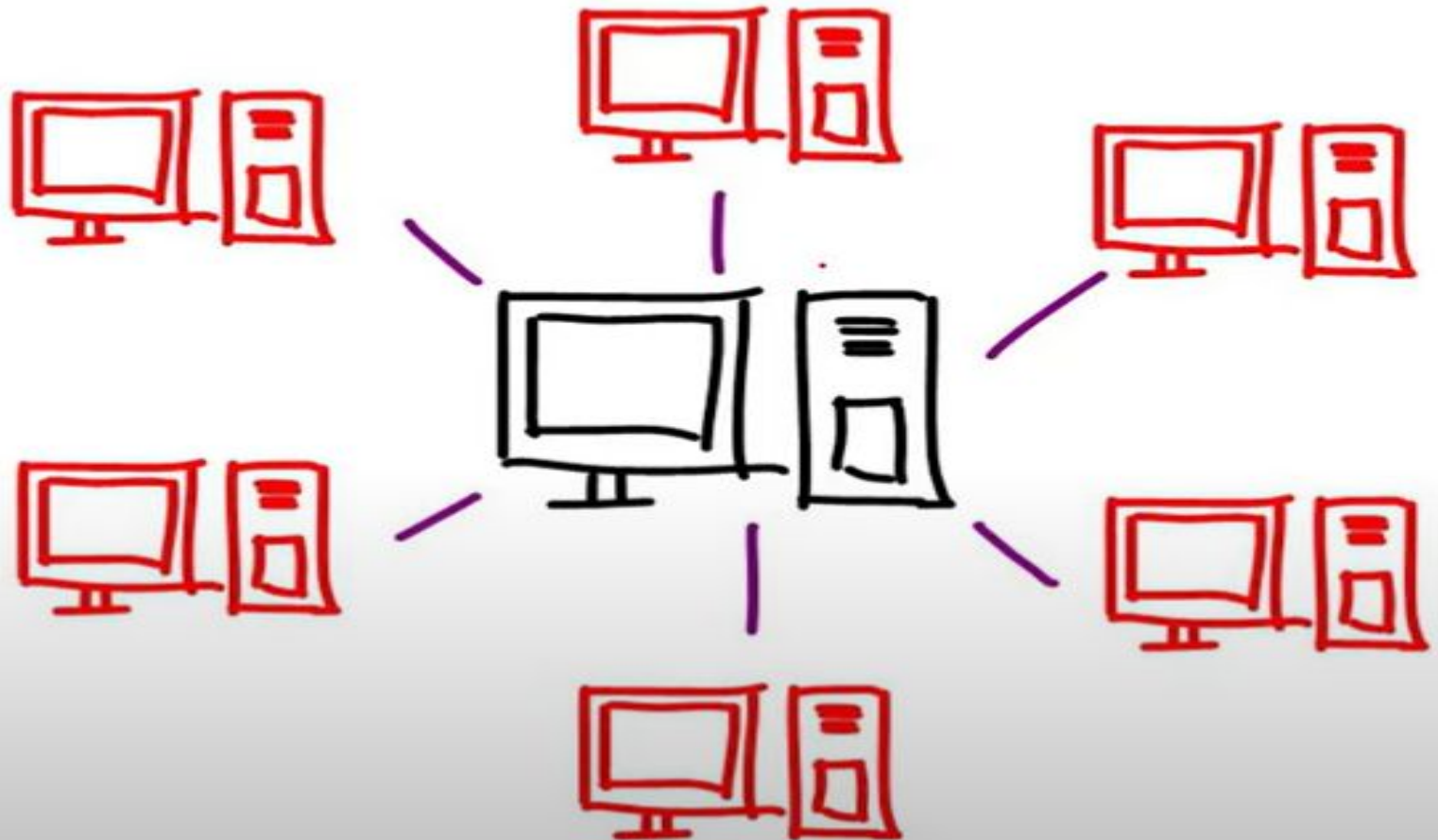
Distributed System

11



Distributed System

12



Why use Parallel Computing?

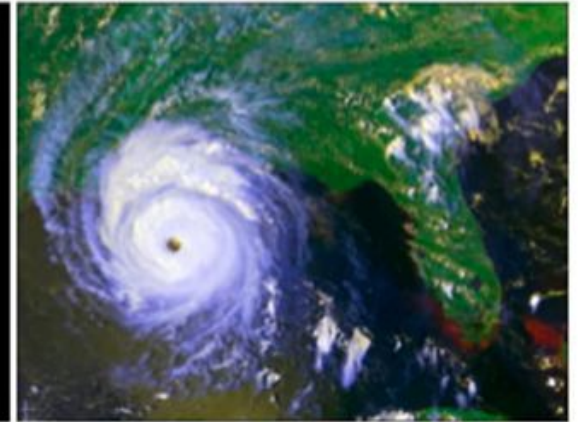
13



Galaxy Formation



Planetary Movments



Climate Change



Rush Hour Traffic



Plate Tectonics



Weather

Activate Wind
Go to Settings to a

Why use Parallel Computing?

14

- Save time and/or money
- In theory, throwing more resources at a task will shorten its time to completion, with potential cost savings.
- Parallel computers can be built from cheap, commodity components.

Why use Parallel Computing?

15

- Solve larger / more complex problems
- Many problems are so large and/or complex that it is impractical or impossible to solve them using a serial program, especially given limited computer memory.
- Example: Web search engines/databases processing millions of transactions every second

Activate Windows

Why use Parallel Computing?

16

- Solve larger / more complex problems
- Many problems are so large and/or complex that it is impractical or impossible to solve them using a serial program, especially given limited computer memory.
- Example: Web search engines/databases processing millions of transactions every second

Activate Windows

Why use Parallel Computing?

17

- Make better use of underlying parallel hardware
- Modern computers, even laptops, are parallel in architecture with multiple processors/cores.
- Parallel software is specifically intended for parallel hardware with multiple cores, threads, etc.
- In most cases, serial programs run on modern computers "waste" potential computing power.

Activate Windows
Go to Settings to activate Windows

Why use Parallel Computing?

- Make better use of underlying parallel hardware
- Modern computers, even laptops, are parallel in architecture with multiple processors/cores.
- Parallel software is specifically intended for parallel hardware with multiple cores, threads, etc.
- In most cases, serial programs run on modern computers "waste" potential computing power.

Activate Windows
Go to Settings to activate Windows

Parallel Computing vs Distributed Computing

19

<u>Sr.</u>	<u>Parallel Computing</u>	<u>Distributed Computing</u>
1.	Many operations are performed simultaneously	System components are located at different locations
2.	Single computer is required	Uses multiple computers
3.	Multiple processors perform multiple operations	Multiple computers perform multiple operations

Parallel Computing vs Distributed Computing

20

<u>Sr.</u>	<u>Parallel Computing</u>	<u>Distributed Computing</u>
1.	Many operations are performed simultaneously	System components are located at different locations
2.	Single computer is required	Uses multiple computers
3.	Multiple processors perform multiple operations	Multiple computers perform multiple operations
4.	It may have shared or distributed memory	It have only distributed memory
5.	Processors communicate with each other through bus	Computer communicate with each other through message passing.
6.	Improves the system performance	Improves system scalability, fault tolerance and resource sharing capabilities

Why not to use Parallel Computing

21

1. Solve larger / more complex problems
2. Provide concurrency
3. Take advantage of non-local resources
4. Make better use of underlying parallel hardware

Motivating Parallelism

Motivating Parallelism

23

- Developing parallel hardware and software has traditionally been time and effort intensive.
- If one is to view this in the context of rapidly improving uniprocessor speeds, one is tempted to question the need for parallel computing.
- Latest trends in hardware design indicate that uni-processors may not be able to sustain the rate of *realizable* performance increments in the future .
- This is the result of a number of fundamental physical and computational limitations.
- The emergence of standardized parallel programming environments, libraries, and hardware have significantly reduced time to develop (parallel) solution.

Need for parallelism

24

- ❑ Advances in Microprocessor technology.
- ❑ Clock rate 40 MHz $\rightarrow$ 2.0GHz
- ❑ Processors capable to run multiple instructions in same cycle
- ❑ Cycles per instructions have increased
- ❑ All of this results in peak floating point operations per second (FLOPS)
- ❑ Ability of memory systems to feed data to processes.

Need for parallelism

25

- Concurrency is one solution but results in problems like **multiplicity of data paths, increased access to storage elements, scalable performance and lower costs.**

Motivating Parallelism

26

Moore's Law

- Proposed by Gordon E. Moore in 1965 and revised in 1975.
- It states that [Simplified Version]
“Processing speeds, or overall processing power for computers will double every 18 months.”
- A more technically correct interpretation
“The number of transistors on an affordable CPU would double every two years [18 months].”

Motivating Parallelism

27

Moore's Law

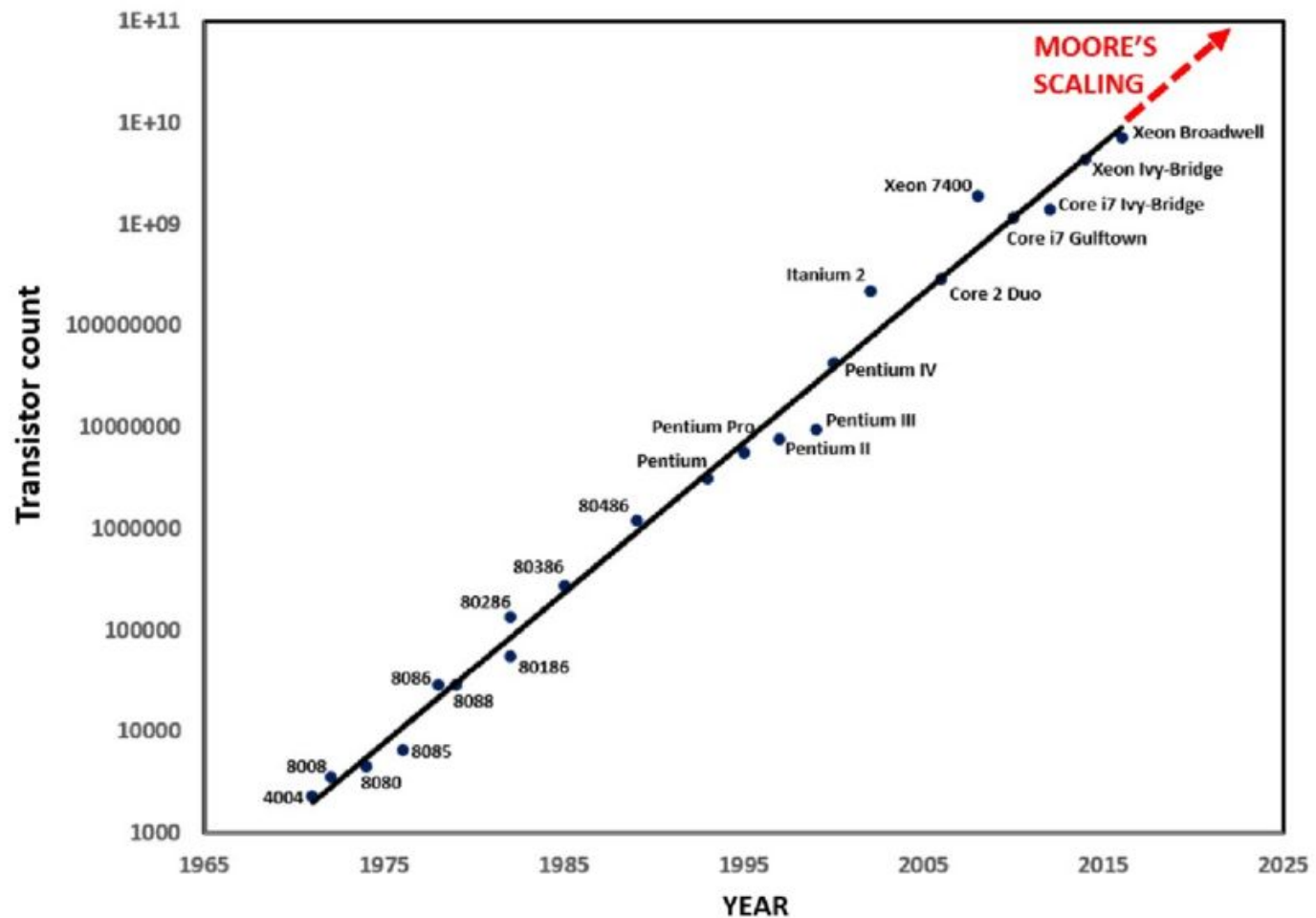
- More computational power implicitly means more transistors.
- Then why need second interpretation?
- Let's have a look on empirical data from 1970 to 2009
 - In 1970's (i.e., from 1970 to 1979), processor speeds ranged from 740 KHz to 8 Mhz. Difference shows that both the interpretations are correct.
 - From 2000 to 2009, Speeds ranged from 1.3 GHz to 2.8 GHz.
 - Speed difference is too low but, number of integrated transistors ranged from 37.5 million to 904 million.
 - So, second interpretation is more accurate.

Motivating Parallelism

28

Moore's Law

- Why doubling the transistors does not doubles the speed?
 - The answer is increase in number of transistor per processor is due to multi-core CPU's.
 - It means, to follow Moore's law, companies had to:
 - Introduce ULSI(ultra large-scale integrations)
 - And multi-core processing era.
- Will Moore's law hold forever?
 - Adding multiple cores on single chip causes heat issues.
 - Furthermore, increasing the number of cores, may not be able to increase speeds [Due to inter-process interactions].
 - Moreover, transistors would eventually reach the limits of miniaturization at atomic levels



Problems with single “Faster” CPU

30

□ Power Consumption:

- With growing clock-frequencies, consumption increases exponentially
- **Not possible forever:** A higher frequency processor could have required a whole nuclear reactor to fulfill the energy needs

Problems with single “Faster” CPU

31

□ Heat Dissipation:

- With growing clock frequency, more power is consumed →
More heat is dissipated..
- Keeping processors cool become a problem (with increasing clock frequencies)

Problems with single “Faster” CPU

32

□ **Smaller size of transistors:**

- Fabrication of processors is difficult

□ **Limited memory size:**

- A single processor has limited internal memory i.e., cache memory L1, L2, registers, etc.
- A single processor-based system has limited overall system memory or main memory

Motivating Parallelism

33

Moore's Law

- So, we must look for efficient parallel software solutions to fulfill our future computational needs.
- As stated earlier, number of cores on a single chip also have some restrictions.
- Solution[s]?
 - Need to find more scalable distributed and hybrid solutions

Motivating Parallelism

34

The Memory/Disk Speed Argument

- While clock rates of high-end processors have increased at roughly 40% per year over the past decade, DRAM access times have only improved at the rate of roughly 10% per year over this interval.
- This mismatch in speeds causes significant performance bottlenecks.
- Parallel platforms provide increased bandwidth to the memory system.
- Parallel platforms also provide higher aggregate **caches**.
- Some of the fastest growing applications of parallel computing utilize not their raw computational speed, rather their ability to pump data to memory and disk faster.

Motivating Parallelism

35

The Data Communication Argument

- As the network evolves, the vision of the Internet as one large computing platform has emerged.
- In many applications like databases and data mining problems, the volume of data is such that they cannot be moved.
- Any analyses on this data must be performed over the network using parallel techniques

Computing vs Systems

36

Distributed Systems

- A collection of autonomous computers, connected through a network and distribution middleware.
 - This enables computers to coordinate their activities and to share the resources of the system.
 - The system is usually perceived as a single, integrated computing facility.
 - Mostly concerned with the hardware-based accelerations

Distributed Computing

- A specific use of distributed systems, to split a large and complex processing into subparts and execute them in parallel, to increase the productivity.
 - Computing mainly concerned with software-based accelerations (i.e., designing and implementing algorithms)

Parallel vs Distributed Computing

37

Parallel (shared-memory) Computing

- The term is usually used for developing concurrent solutions for following two types of the systems:
 1. Multi-core Architecture
 2. Many core architectures (i.e., GPU's)

Distributed Computing

- This type of computing is mainly concerned with developing algorithms for the distributed cluster systems.
- Here distributed means a geographical distance between the computers without any shared-Memory.

Practical Applications of P&D Computing

38

Scientific Applications

- Functional and structural characterization of genes and proteins
- Applications in astrophysics have explored the evolution of galaxies, thermonuclear processes, and the analysis of extremely large datasets from telescope.
- Advances in computational physics and chemistry have explored new materials, understanding of chemical pathways, and more efficient processes
 - e.g., Large Hydron Collider (LHC) at European Organization for Nuclear Research (CERN) generates petabytes of data for a single collision.

Practical Applications of P&D Computing

39

Scientific Applications

- Bioinformatics and astrophysics also present some of the most challenging problems with respect to analyzing extremely large datasets.
- Weather modeling for simulating the track of a natural hazards like the extreme cyclones (storms).
- Flood prediction

Practical Applications of P&D Computing

40

Commercial Applications

- Some of the largest parallel computers power the wall street!
- Data mining-analysis for optimizing business and marketing decisions.
- Large scale servers (mail and web servers) are often implemented using parallel platforms.
- Applications such as information retrieval and search are typically powered by large clusters.

Practical Applications of P&D Computing

41

Computer Systems Applications

- Network intrusion detection: A large amount of data needs to be analyzed and processed
- Cryptography (the art of writing or solving codes) employs parallel infrastructures and algorithms to solve complex codes.
- Graphics processing
- Embedded systems increasingly rely on distributed control algorithms.

Limitations of Parallel Computing:

42

- It requires designing the proper communication and synchronization mechanisms between the processes and sub-tasks.
- Exploring the proper parallelism from a problem is a hectic process.
- The program must have low coupling and high cohesion. But it's difficult to create such programs.
- It needs relatively more technical skills to code a parallel program.

Organization and Contents of the Course

43

- **Fundamentals:** This part of the class covers basic parallel platforms, principles of algorithm design, group communication primitives, and analytical modeling techniques.
- **Parallel Programming:** This part of the class deals with programming using message passing libraries, threads and the kernels.
- **Parallel Algorithms:** This part of the class covers basic algorithms for matrix computations, graphs, sorting, and optimization using dynamic programming.

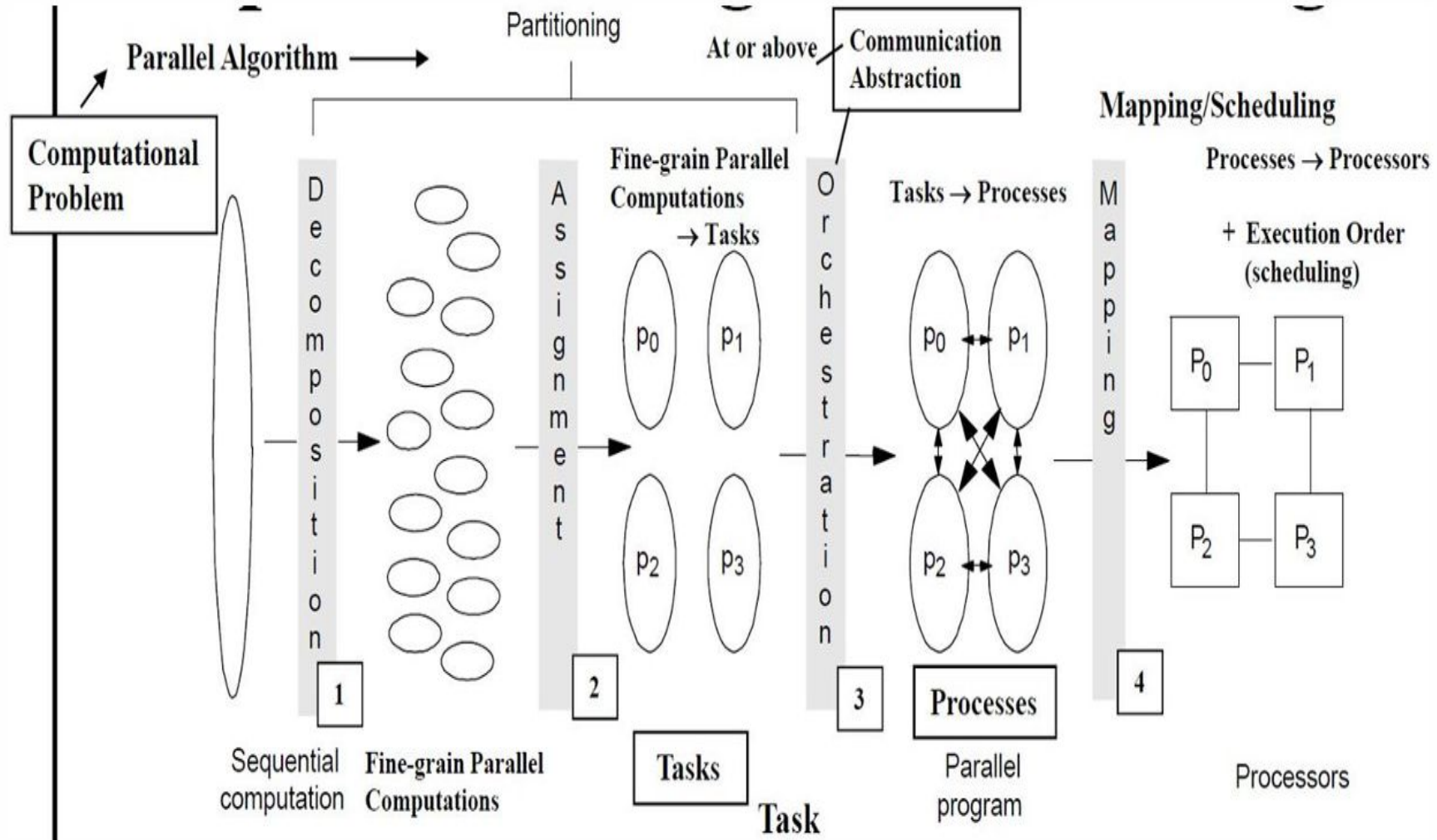
Parallelization Strategy

44

- Problem Understanding
- Partitioning/Decomposition
- Assignment
- Orchestration
- Mapping

Parallelization Strategy - Overview

45



Parallelization Strategy

46

- **Problem Understanding**
- Partitioning/Decomposition
- Assignment
- Orchestration
- Mapping

1. Problem Understanding

47

- The first step in developing parallel application is to Understand the Problem (that you wish to solve)
- Starting with a serial program → Understanding the existing code
- The first decision → Whether or not the problem is suitable for parallelization?
 - Code Dependencies
 - Communication
 - Synchronization

1. (a) Identify the Programs hot-spots

48

- Know where most of the real work is being done. (The majority of scientific and technical programs usually accomplish most of their work in a few places.)
- Profilers and performance analysis tools can help here
- Focus on parallelizing the hotspots and ignore those sections of the program that account for little CPU usage

1. (a) Identify the Programs hot-spots

49

- Identify areas that are unjustly slow?
 - Example: I/O is usually something that slows a program
- Potential Solutions:
 - Restructure the program
 - Use a different algorithm
 - Overlap the communication with computation

2. Partitioning/Decomposition

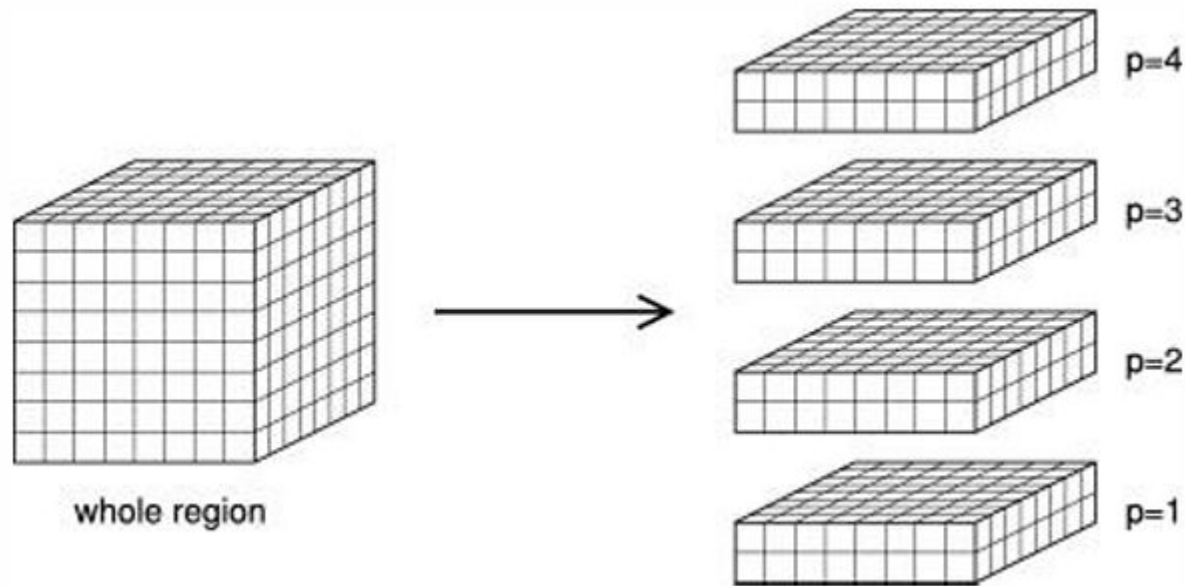
50

- Divide work into discrete "chunks"
- Broken chunks can be assigned to specific tasks
- Tasks could be executed concurrently
- **Decomposition:**
 1. Domain decomposition
 2. Functional decomposition
- **Decomposition Granularity:**
 - Fine-grain: Large number of small tasks
 - Coarse-grain: small number of large tasks

2. (a) Domain Decomposition

51

- The data associated with a problem is decomposed
- Each parallel task then works on a portion of the data



2. (b) Functional Decomposition

52

- Focus is on the computation that is to be performed
- The problem is decomposed according to the work (instructions) that must be done
- Each sub-task performs a portion of the overall work
- Functional decomposition useful to the problems that can be split into different tasks
- Examples:
 - Ecosystem Modeling
 - Signal Processing
 - Climate Modeling

3. Assignment

53

- Composing “Fine-grained” parallel computations into processes or coarse-grained tasks
- **Considerations while composing tasks:** Load-balance, Uniform communication, ease of Synchronization

	Fine-Grain Parallelism	Coarse-Grain Parallelism
Computation/Communication ratio	Low	High
Computational work between communication stages	Small	Large
Communication/Synchronization overhead	High	Low
Load balance efficiency	High	Low

4-5. Orchestration & Mapping

54

□ Following aspects should be considered before actual mapping (Scheduling):

1. Inter-task communication
2. Synchronization among tasks
3. Data locality aspect
4. Other system related considerations etc.).

Communication Cost

55

- ❑ Inter-task communication virtually always implies overhead
- ❑ CPU time and resources are used to package and transmit data
- ❑ Communications mostly require some synchronization between tasks → Results in waiting time
- ❑ Communication can saturate the available network bandwidth, further reduce performance

Latency & Bandwidth

56

- ❑ **Latency** is the time it takes to send a message from point A to point B
- ❑ **Bandwidth** is the amount of data that can be communicated per unit of time
- ❑ Sending many small messages can cause a higher overall latency
- ❑ Efficient approach is to package small messages into a larger message
 - To maximize bandwidth usage

Questions



57